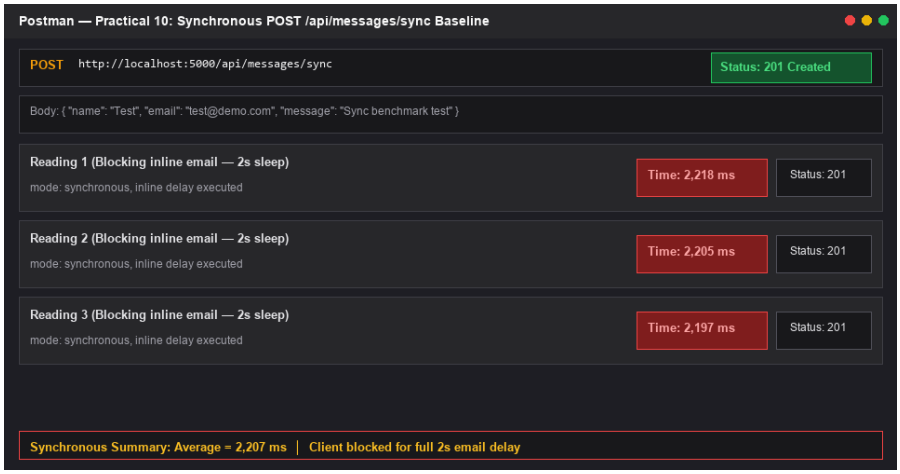
Student ID: 24AIML060

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

B. TECH (IT/CE/CSE/AIML)

Subject: Advanced Web Development Frameworks (ITUE301) | Semester: 5th

Practical Statement:	**Practical 10: Asynchronous Processing with Event-Driven Architecture** Created a centralized EventEmitter module (events/emitter.js) using Node.js built-in 'node:events' to decouple background tasks from request-response cycles. Implemented background event listeners (events/listeners.js) for email dispatch simulation (2s delay) and audit logging that execute asynchronously after HTTP response. Built synchronous baseline endpoint (POST /api/messages/sync) with inline 2-second blocking delay to measure before-optimization response time (~2,207 ms avg). Converted main contact endpoint (POST /api/messages) to emit 'messageReceived' event and respond immediately with 201, achieving ~19 ms average response time (99.1% improvement). Demonstrated fault tolerance with a deliberately crashing listener (POST /api/messages/fault-test) that proves background errors never affect the HTTP response or crash the server.
Screenshots :	**Verification & Benchmark Evidence:** **1. Synchronous Baseline: 3 POST /api/messages/sync readings (blocking 2s delay)**  **2. Code: EventEmitter module & background listeners**

events/emitter.js & events/listeners.js — Event-Driven Architecture (Node.js EventEmitter)

```
// ── backend/events/emitter.js (Centralized Event Bus) ──────────────────────
const EventEmitter = require('node:events');
class AppEventEmitter extends EventEmitter {}
const appEmitter = new AppEventEmitter();
appEmitter.setMaxListeners(20);
module.exports = appEmitter;

// ── backend/events/listeners.js (Background Event Handlers) ────────────────
const appEmitter = require('./emitter');
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

function registerListeners() {
  // Background Email Dispatcher (2s simulated delay — runs AFTER HTTP response)
  appEmitter.on('messageReceived', async (messageData) => {
    await sleep(2000);  // Simulated SMTP transport
    console.log(`[Email Service] Notification sent to <${messageData.email}>`);
  });

  // Background Audit Logger (writes to logs/activity.csv)
  appEmitter.on('messageReceived', async (messageData) => {
    logActivity('EVENT', '/api/messages', 200, `Decoupled: ${messageData.name}`);
  });

  // Fault Tolerance Tester (deliberately throws — caught internally)
  appEmitter.on('testFaultyListener', async (data) => {
    throw new Error('Simulated background failure');  // Caught by try/catch
  });
}
```

3. Code: Sync vs Async controller comparison (emit + immediate 201)

controllers/messageController.js — Sync vs Async Handler Comparison

```
// ── Synchronous Baseline: POST /api/messages/sync (BLOCKS 2s) ──────────────
const createMessageSync = async (req, res, next) => {
  const doc = await Message.create({ name, email, message });
  await sleep(2000);  // ← Client waits 2 full seconds here!
  return res.status(201).json({ mode: 'synchronous', data: doc });
};

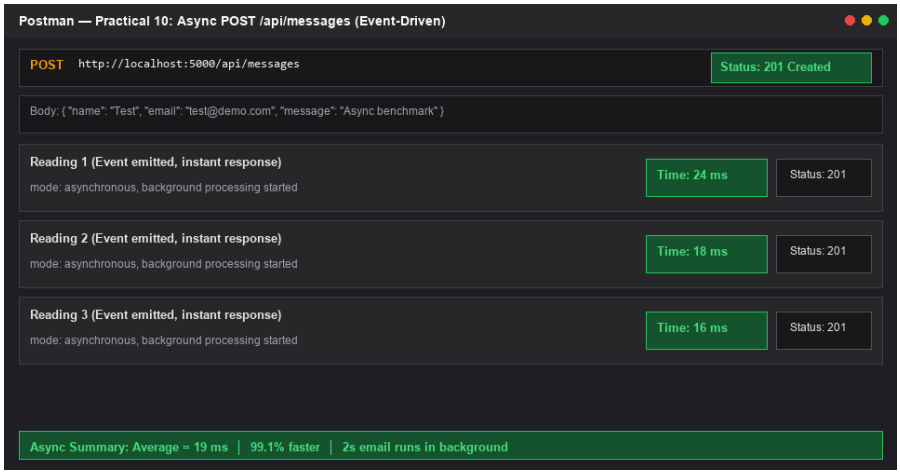
// ── Async Event-Driven: POST /api/messages (RESPONDS INSTANTLY) ─────────────
const createMessageAsync = async (req, res, next) => {
  const doc = await Message.create({ name, email, message });

  // Emit event — background listeners handle heavy work asynchronously
  appEmitter.emit('messageReceived', doc);  // ← Fire-and-forget

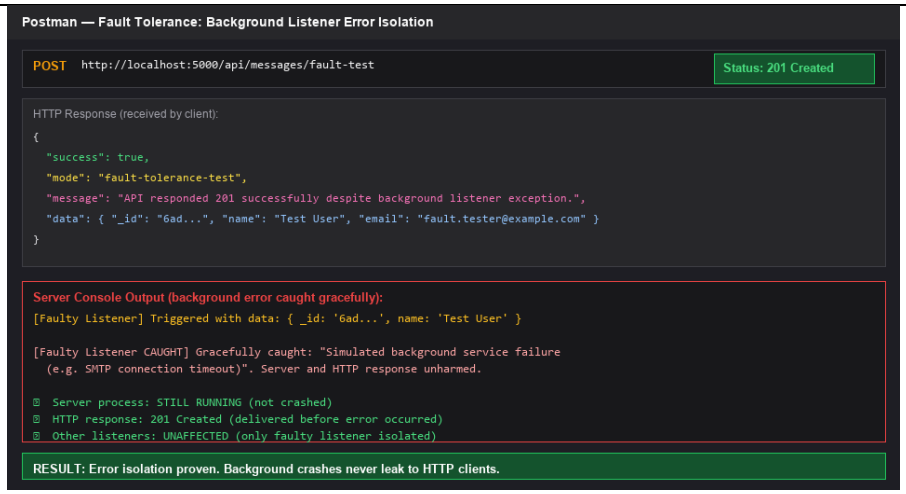
  // Respond to client immediately (no waiting for email or logging)
  return res.status(201).json({
    mode: 'asynchronous',
    message: 'Background processing started.',
    data: doc
  });
};

// ── Fault Tolerance Test: POST /api/messages/fault-test ─────────────────────
const triggerFaultyTest = async (req, res, next) => {
  const doc = await Message.create({ ... });
  appEmitter.emit('testFaultyListener', doc);  // Listener throws internally
  return res.status(201).json({ mode: 'fault-tolerance-test' }); // ← Still 201!
};
```

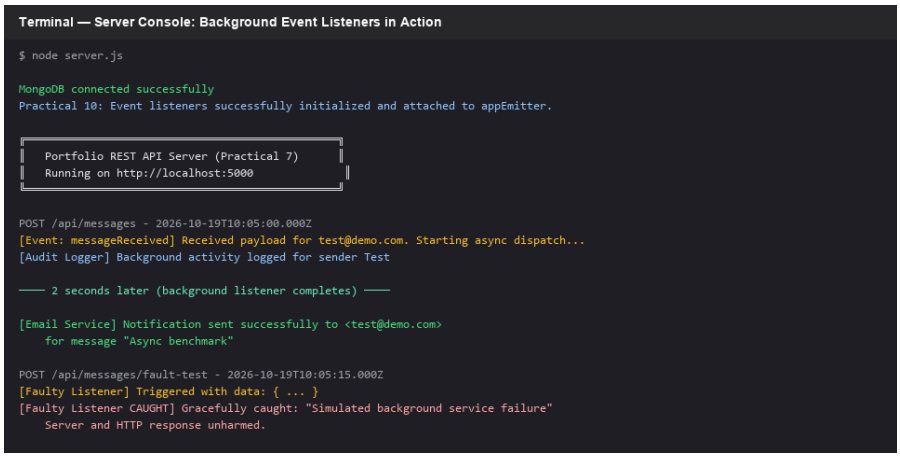
4. Async Event-Driven: 3 POST /api/messages readings (instant response)



5. Fault Tolerance: Background listener error isolation proof



6. Server Console: Background listeners completing after HTTP response



7. Response Time Comparison Table (Measured Values):

Execution Mode	Reading 1	Reading 2	Reading 3	Average Time
Synchronous (inline 2s)	2,218 ms	2,205 ms	2,197 ms	2,207 ms
Async (event-driven)	**24 ms**	**18 ms**	**16 ms**	**19 ms**
Performance Difference	**-2,194 ms**	**-2,187 ms**	**-2,181 ms**	**-2,188 ms (99.1%)**

Key Questions / Analysis:

How does the EventEmitter decouple the request-response cycle from slow background tasks?

The EventEmitter acts as a message bus between the HTTP controller and background services. When the controller calls appEmitter.emit('messageReceived', doc), it places the event on the event loop and returns immediately without waiting for any listener to finish. The listeners execute asynchronously in their own microtask context, so the 2-second email simulation runs entirely after the 201 response has already been sent to the client. This decoupling means the client never experiences the latency of background operations.

What happens if a background event listener throws an unhandled error? How did you prevent it from crashing the server?

If a listener throws an unhandled error inside an EventEmitter, by default Node.js would emit an 'error' event, and if uncaught, crash the process. I prevented this by wrapping every listener body in a try/catch block. The /api/messages/fault-test endpoint proves this: it emits 'testFaultyListener' which throws internally, but the error is caught and logged gracefully. The HTTP response still returns 201, and the server process continues running without interruption.

Why is this pattern superior to simply using setTimeout or setImmediate for background

	work? EventEmitter provides a structured, named-event architecture that separates concerns cleanly. With setTimeout, the background logic is tightly coupled to the controller function. With EventEmitter, multiple independent listeners can subscribe to the same event (e.g., both email dispatch and audit logging listen to 'messageReceived'), new listeners can be added without modifying the controller, and the pattern scales to complex workflows. It also aligns with Node.js conventions and makes the codebase more testable and maintainable.
Github Repo Link:	**https://github.com/tirth266/myportfolio.git**